LAPORAN TUGAS PERKULIAHAN
PENGEMBANGAN SISTEM BERBASIS WEB_14823293



Kelas : R

Gara Zaki Pramudya 1482500069

Dosen Pengampu

Abdullah Faqih Septiyanto, S.Kom., M.Kom.

PROGRAM STUDI SISTEM TEKNOLOGI & INFORMASI

OKTOBER 2026

Tugas Individu 2: Website Interaktif (DOM, Event, dan jQuery)

1. Gambaran Umum

Tugas ini terdiri dari tiga file yang berada dalam satu folder:

File	Fungsi
`index.html`	Kerangka halaman: judul, kolom cari, tombol, form, dan wadah kosong untuk daftar menu.
`style.css`	Tampilan: font, ukuran, jarak, garis kartu, dan warna pesan error.
`script.js`	Logika: menyimpan data, membuat tampilan lewat DOM, serta menangani event dan jQuery.

Ide utama

Data berubah, lalu tampilkan ulang. Halaman web tidak mengingat tampilan. Yang disimpan hanya array items. Setiap kali array berubah (tambah, hapus, cari), semua kartu digambar ulang dari array tersebut.

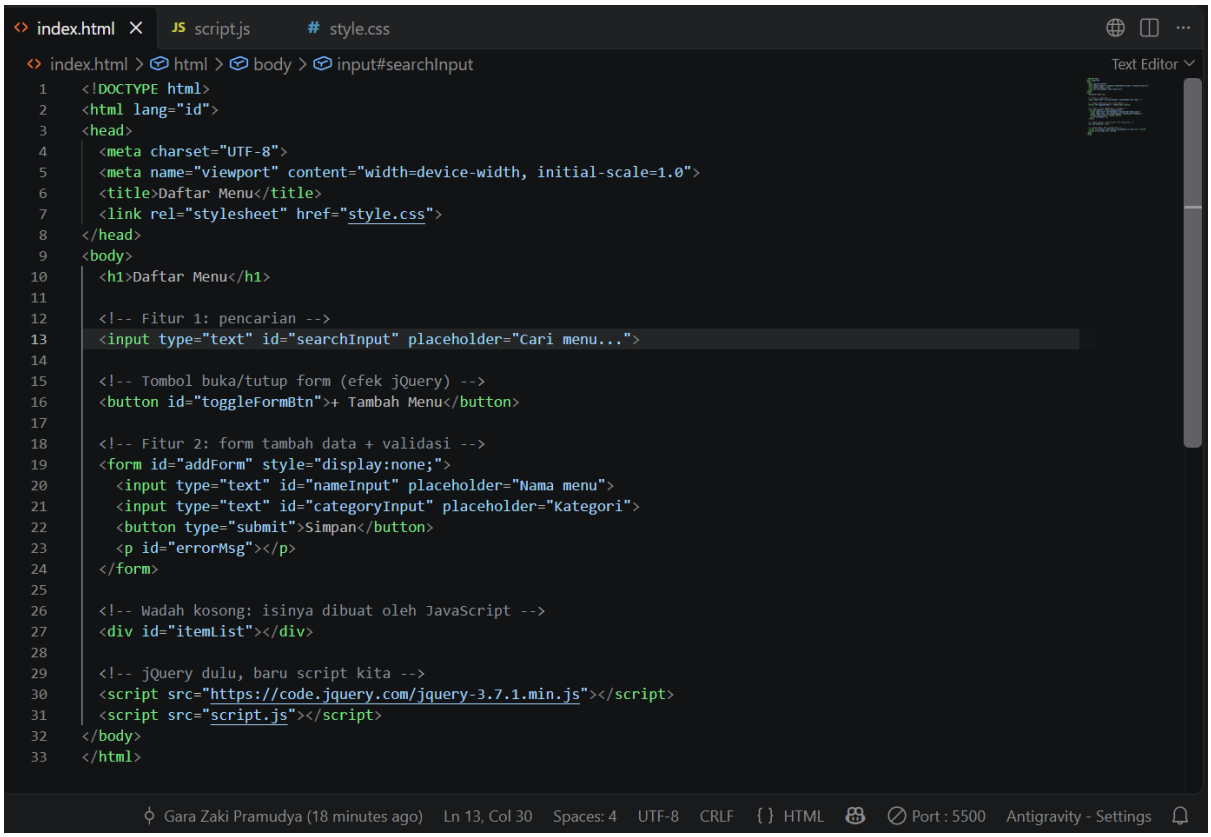
Analogi: array items adalah papan tulis, sedangkan kartu di layar adalah foto papan tulis. Jika papan tulis diubah, kita mengambil foto baru, bukan mengedit fotonya.

Karena itu, menu yang diketik langsung di file HTML tidak akan bereaksi. Menu harus ditambahkan ke array `items` di script.js atau lewat form.

2. Penjelasan index.html

File ini hanya berisi kerangka. Perhatikan bahwa tidak ada satu pun menu yang ditulis di sini; daftar menu dibuat oleh JavaScript.

Kode lengkap



Penjelasan

Kode	Penjelasan
`<!DOCTYPE html>`	Memberi tahu browser bahwa dokumen ini memakai standar HTML5.
`<html lang="id">`	Pembuka dokumen HTML. Atribut lang="id" menandakan bahasa halaman adalah Indonesia.
`<meta charset="UTF-8">`	Pengaturan penulisan karakter supaya huruf dan simbol tampil benar.
`<meta name="viewport" content="width=device-width, initial-scale=1.0">`	Membuat halaman menyesuaikan lebar layar, sehingga tampil baik di HP.
`<title>Daftar Menu</title>`	Judul yang tampil di tab browser.
`<link rel="stylesheet" href="style.css">`	Menghubungkan file CSS ke halaman ini.
`<h1>Daftar Menu</h1>`	Judul utama di halaman.

Kode	Penjelasan
`<input type="text" id="searchInput" placeholder="Cari menu...">`	Kolom pencarian (Event 1). id="searchInput" adalah "nama panggilan" yang dipakai JavaScript untuk menemukan elemen ini. placeholder adalah teks petunjuk abu-abu.
`<button id="toggleFormBtn"> + Tambah Menu</button>`	Tombol untuk membuka dan menutup form. Efeknya dikerjakan jQuery (slideToggle).
`<form id="addForm" style="display:none;">`	Form tambah menu (Event 2). display:none membuat form tersembunyi saat halaman pertama dibuka. Baru muncul setelah tombol Tambah Menu diklik.
`<input type="text" id="nameInput" placeholder="Nama menu">`	Kolom isian nama menu. Nilainya dibaca JavaScript lewat nameInput.value.
`<input type="text" id="categoryInput" placeholder="Kategori">`	Kolom isian kategori. Nilainya dibaca lewat categoryInput.value.
`<button type="submit"> Simpan</button>`	Tombol kirim. type="submit" memicu event submit pada form.
`<p id="errorMsg"></p>`	Tempat menampilkan pesan error validasi. Awalnya kosong, diisi oleh JavaScript.
`<div id="itemList"></div>`	Wadah kosong. Seluruh kartu menu dibuat dan dimasukkan ke sini oleh JavaScript (syarat nomor 1: data ditampilkan lewat DOM).
`<script src="https://code.jquery.com/ jquery-3.7.1.min.js"></script>`	Memuat library jQuery dari internet (CDN). Harus ditulis sebelum script.js, karena script.js memakai tanda $.
`<script src="script.js"></script>`	Memuat file JavaScript kita. Ditaruh paling bawah supaya semua elemen HTML sudah ada saat script berjalan.

3. Penjelasan style.css

CSS mengatur tampilan. Pola penulisannya: pemilih (selector), lalu properti dan nilainya di dalam kurung kurawal.

Kode lengkap



Penjelasan

Kode	Penjelasan
`body { ... }`	Berlaku untuk seluruh isi halaman. font-family: Arial, sans-serif; memakai font Arial. max-width: 600px; lebar maksimum konten 600 piksel. margin: 30px auto; jarak atas-bawah 30px, kiri-kanan otomatis sehingga konten berada di tengah. padding: 0 15px; jarak dalam kiri-kanan 15px agar teks tidak menempel ke tepi layar.
`input { ... }`	Berlaku untuk semua kolom input. padding: 8px; ruang di dalam kolom agar lebih lega. margin: 4px 0; jarak atas-bawah antar kolom.
`button { ... }`	Berlaku untuk semua tombol. padding: 8px 12px; ruang di dalam tombol. cursor: pointer; kursor berubah menjadi tangan saat di atas tombol.
`.card { ... }`	Titik (.) berarti class. Berlaku untuk elemen ber-class "card", yaitu kartu yang dibuat oleh renderItems di JavaScript (card.className = "card"). border: 1px solid ...; garis tepi tipis. border-radius: 8px; sudut membulat. padding: 10px 15px; ruang di dalam kartu. margin: 10px 0; jarak antar kartu.
`#errorMsg { ... }`	Tanda pagar (#) berarti id. Berlaku untuk elemen id="errorMsg". color: red; pesan error berwarna merah.

4. Penjelasan script.js

File ini adalah inti tugas. Kodenya dibagi menjadi beberapa bagian sesuai komentar di dalam file.

Kode lengkap

index.html | script.js | style.css

```
// ===== 1. DATA (array of object) =====
let items = [
  { id: 1, name: "Nasi Goreng", category: "Makanan" },
  { id: 2, name: "Es Teh", category: "Minuman" },
  { id: 3, name: "Mie Ayam", category: "Makanan" },
  { id: 4, name: "Menu Baru Kamu", category: "Makanan" }
];

// ===== 2. AMBIL ELEMEN DOM =====
const itemList = document.getElementById("itemList");
const searchInput = document.getElementById("searchInput");
const addForm = document.getElementById("addForm");
const nameInput = document.getElementById("nameInput");
const categoryInput = document.getElementById("categoryInput");
const errorMsg = document.getElementById("errorMsg");

// ===== 3. TAMPILKAN DATA LEWAT DOM =====
function renderItems(list) {
  itemList.innerHTML = "";

  if (list.length === 0) {
    itemList.textContent = "Data tidak ditemukan.";
    return;
  }

  list.forEach(function (item) {
    // buat elemen secara manual lewat JS
    const card = document.createElement("div");
    card.className = "card";

    const title = document.createElement("h3");
    title.textContent = item.name;

    const category = document.createElement("p");
```

Gara Zaki Pramudya (21 minutes ago) Ln 95, Col 1 Spaces: 2 UTF-8 CRLF { } JavaScript Port : 5500 Antigravity - Settings

```
    category.textContent = item.category;

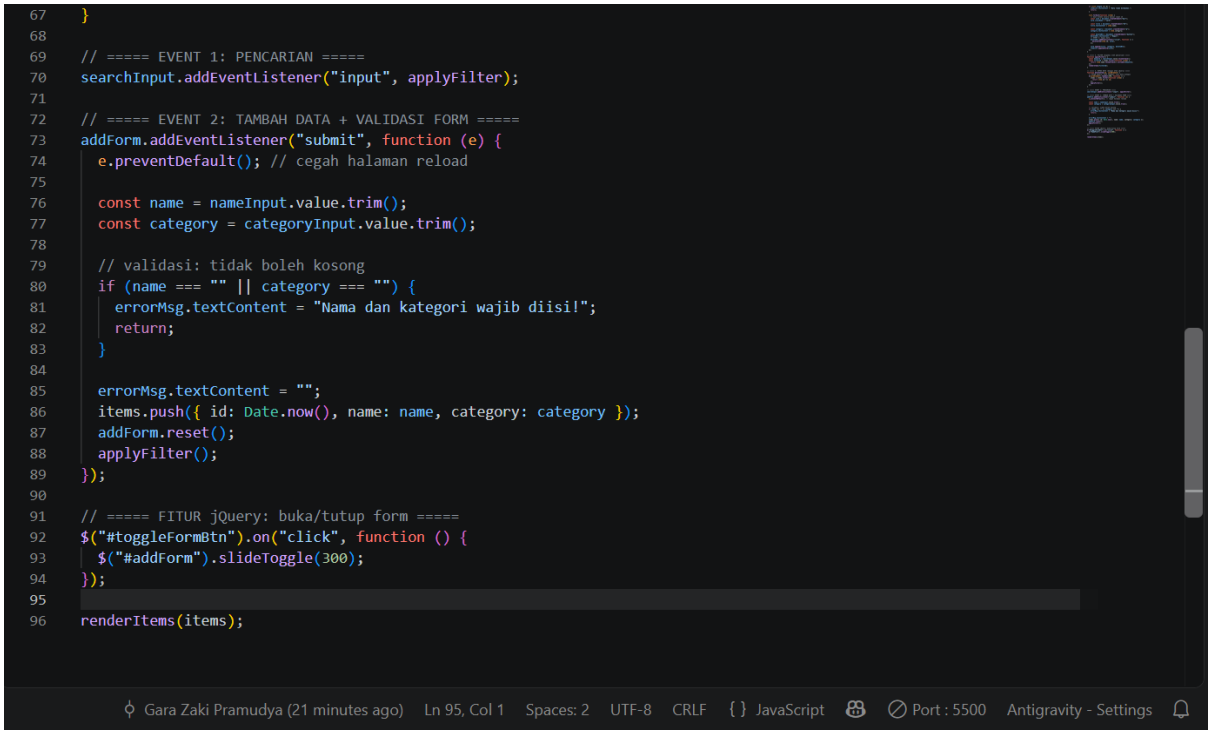
    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Hapus";
    // EVENT 3: hapus data
    deleteBtn.addEventListener("click", function () {
      deleteItem(item.id, card);
    });

    card.append(title, category, deleteBtn);
    itemList.appendChild(card);
  });
}

// ===== 4. FILTER (dipakai oleh pencarian) =====
function applyFilter() {
  const keyword = searchInput.value.toLowerCase();
  const filtered = items.filter(function (item) {
    return item.name.toLowerCase().includes(keyword);
  });
  renderItems(filtered);
}

// ===== 5. HAPUS DATA (dengan efek jQuery) =====
function deleteItem(id, cardElement) {
  // FITUR jQuery: efek fadeOut, lalu data dihapus
  $(cardElement).fadeOut(300, function () {
    items = items.filter(function (item) {
      return item.id !== id;
    });
    applyFilter();
  });
```

Gara Zaki Pramudya (21 minutes ago) Ln 95, Col 1 Spaces: 2 UTF-8 CRLF { } JavaScript Port : 5500 Antigravity - Settings



Bagian 1: Data (array of object)

Kode	Penjelasan
`let items = [ ... ];`	let membuat variabel yang nilainya boleh diganti. Dipilih let (bukan const) karena isi daftar akan berubah saat tambah dan hapus data. [ ] adalah array, yaitu daftar berurutan.
`{ id: 1, name: "Nasi Goreng", category: "Makanan" }`	{ } adalah object, yaitu satu data dengan beberapa properti: id, name, category. Properti diakses dengan titik, misalnya item.name menghasilkan "Nasi Goreng". id adalah penanda unik. Dipakai saat menghapus agar jelas data mana yang dimaksud.

Bagian 2: Mengambil elemen DOM

Kode	Penjelasan

`const itemList =`
 	
`document.getElementById("itemList");`
	JavaScript tidak otomatis tahu isi HTML, jadi elemen harus "ditunjuk" dulu.
	document = seluruh halaman. getElementById = cari elemen berdasarkan id. Hasilnya disimpan di variabel agar bisa dipakai berulang.
	Enam baris di bagian ini melakukan hal yang sama untuk elemen berbeda (searchInput, addForm, nameInput, categoryInput, errorMsg).
	Penting: teks di dalam tanda kutip harus persis sama dengan id di HTML. Salah satu huruf saja akan membuat script berhenti.

Bagian 3: renderItems (menampilkan data lewat DOM)

Fungsi ini mengubah array menjadi tampilan. Alurnya: kosongkan wadah, ulangi untuk setiap data, buat kartu, lalu tempel ke halaman.

Kode	Penjelasan
`function renderItems(list) {`	Mendefinisikan fungsi. Parameter list adalah daftar data yang akan ditampilkan (bisa seluruh data, bisa hasil pencarian).
`itemList.innerHTML = "";`	Mengosongkan wadah. Tanpa ini, setiap render akan menumpuk di bawah kartu lama sehingga kartu muncul dobel.

`if (list.length === 0) {`
`  itemList.textContent =`
`  "Data tidak ditemukan.";`
`  return;`
`}`
	Jika daftar kosong (misalnya hasil pencarian tidak ada), tampilkan pesan lalu return untuk menghentikan fungsi.

`list.forEach(function (item) {`	forEach artinya "untuk setiap data di daftar, lakukan hal berikut". item mewakili satu data.

`const card =`
`document.createElement("div");`
`card.className = "card";`
	createElement membuat elemen HTML baru lewat kode (inilah penerapan DOM). className memberi class "card" agar terkena CSS .card.

`const title =`
`document.createElement("h3");`
`title.textContent = item.name;`
	Membuat judul h3 dan mengisinya dengan nama menu dari data.

`const category =`
`document.createElement("p");`
`category.textContent =`
`item.category;`
	Membuat paragraf berisi kategori dari data.

Kode	Penjelasan
`const deleteBtn = document.createElement("button"); deleteBtn.textContent = "Hapus";`	Membuat tombol Hapus.
`deleteBtn.addEventListener( "click", function () { deleteItem(item.id, card); });`	Event 3: saat tombol Hapus diklik, jalankan deleteItem dengan membawa id data dan elemen kartunya.
`card.append(title, category, deleteBtn);`	Memasukkan judul, kategori, dan tombol ke dalam kartu.
`itemList.appendChild(card);`	Menempelkan kartu ke halaman. Baru di baris inilah kartu terlihat.

Bagian 4: applyFilter (pencarian)

Kode	Penjelasan
`const keyword = searchInput.value.toLowerCase();`	Mengambil teks yang sedang diketik di kolom cari, lalu diubah menjadi huruf kecil agar "NASI" dan "nasi" dianggap sama.
`const filtered = items.filter(function (item) { return item.name.toLowerCase() .includes(keyword); });`	filter menyaring data: hanya data yang mengembalikan true yang masuk ke daftar baru. includes(keyword) memeriksa apakah nama menu mengandung kata yang diketik. Array items tidak berubah. Yang dibuat adalah daftar baru bernama filtered.
`renderItems(filtered);`	Menampilkan daftar hasil saringan. Jika kolom cari dikosongkan, semua data muncul lagi karena items masih utuh.

Bagian 5: deleteItem (hapus data dengan jQuery)

Kode	Penjelasan
`$(cardElement).fadeOut(300, function () { ... });`	Tanda $ adalah jQuery. $(cardElement) membungkus kartu agar bisa diberi efek. fadeOut(300) membuat kartu memudar selama 300 milidetik. Fungsi di dalamnya baru berjalan setelah efek selesai.
`items = items.filter( function (item) { return item.id !== id; });`	Cara menghapus dari array: simpan semua data kecuali yang id-nya sama dengan yang dihapus (!== artinya "tidak sama dengan"). Hasilnya menimpa items, itulah alasan items memakai let.
`applyFilter();`	Setelah array berubah, tampilkan ulang daftar.

Bagian 6: Event pencarian

Kode	Penjelasan
`searchInput.addEventListener( "input", applyFilter);`	Pola event: elemen.addEventListener("jenis event", fungsi). Dibaca: "pada kolom cari, ketika isinya berubah (input), jalankan applyFilter". Karena itu pencarian langsung bereaksi tiap satu huruf diketik.

Bagian 7: Event tambah data dan validasi form

Kode	Penjelasan
`addForm.addEventListener( "submit", function (e) {`	Event submit berjalan saat form dikirim (tombol Simpan diklik atau Enter ditekan).
`e.preventDefault();`	Mencegah perilaku bawaan form, yaitu memuat ulang halaman.
`const name = nameInput.value.trim(); const category = categoryInput.value.trim();`	Mengambil isi input. trim() membuang spasi di awal dan akhir, sehingga isian yang hanya berisi spasi dianggap kosong.
`if (name === "" || category === "") { errorMsg.textContent = "Nama dan kategori wajib diisi!"; return; }`	Ini adalah validasi form. || artinya "atau": jika salah satu kosong, tampilkan pesan error. return menghentikan fungsi, sehingga baris push di bawahnya tidak dijalankan dan data kosong tidak masuk.
`errorMsg.textContent = "";`	Jika lolos validasi, hapus pesan error.
`items.push({ id: Date.now(), name: name, category: category });`	push menambahkan object baru ke akhir array. Date.now() menghasilkan angka waktu saat ini, dipakai sebagai id yang unik.
`addForm.reset();`	Mengosongkan isian form.
`applyFilter();`	Menampilkan ulang daftar yang sudah memuat data baru.

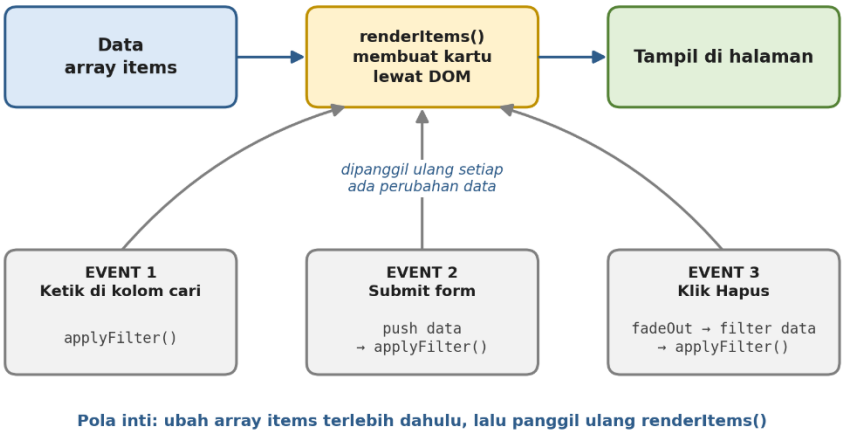
Bagian 8: jQuery buka/tutup form

Kode	Penjelasan
`$("#toggleFormBtn")` `.on("click", function () {`	$("#toggleFormBtn") memilih elemen ber-id toggleFormBtn (# artinya id). .on("click", ...) adalah versi jQuery dari addEventListener.
`$("#addForm").slideToggle(300);`	Efek animasi: form meluncur terbuka jika tertutup, dan meluncur tertutup jika terbuka, selama 300 milidetik.

Bagian 9: Menjalankan pertama kali

Kode	Penjelasan
`renderItems(items);`	Fungsi hanya berjalan jika dipanggil. Tanpa baris ini, halaman kosong saat pertama dibuka.

5. Alur Kerja Keseluruhan



Pola inti: ubah array items terlebih dahulu, lalu panggil ulang renderItems()

Pola intinya: setiap kali data berubah, ubah array terlebih dahulu, lalu panggil ulang renderItems.

6. Link GitHub

[JQueryhttps://github.com/GaraNEIW/Tugas-Individu-2--Pertemuan-6-JQuery]